

Assignment 3 Report

AI 2000

Desaboina Sri Sathwik
Roll No: AI24BTECH11007
Indian Institute of Technology Hyderabad

November 6, 2025

Contents

1	Q3: Spam Classification using Logistic Regression	2
1.1	Objective	2
1.2	Dataset Details	2
1.3	Implementation	2
1.4	Experimental Setup	2
1.5	Summary	3
1.6	Conclusion	3
2	Q4: Autograd Visualization	3
2.1	Objective	3
2.2	Graph Visualization	3
2.3	Explanation	4
2.4	Conclusion	4

1 Q3: Spam Classification using Logistic Regression

1.1 Objective

The objective of this experiment is to implement and evaluate a **Logistic Regression** model from scratch for binary classification on the **Spambase** dataset. The goal is to classify emails as spam or non-spam using features derived from the dataset, and to compare the model performance on both raw and standardized input data.

1.2 Dataset Details

The Spambase dataset consists of 4601 samples and 57 features. Each feature represents the frequency of specific words or characters in email text. The target variable $y \in \{0, 1\}$ indicates whether an email is spam (1) or not spam (0).

$$X \in R^{4601 \times 57}, \quad y \in R^{4601}$$

1.3 Implementation

The logistic regression model was implemented manually within the custom library `my_ml_lib.linear_model`. The hypothesis function is given by:

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^T x)}}$$

and the loss function used is the **Binary Cross-Entropy (Log Loss)**:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Model training was done using gradient descent optimization. A small regularization parameter α (learning rate) was tuned using cross-validation.

1.4 Experimental Setup

Two experiments were conducted:

- **Raw Data:** Model trained on unscaled features.
- **Standardized Data:** Features were standardized to zero mean and unit variance before training.

For each setup, multiple values of α were evaluated using cross-validation. The best performing α value was then used to compute training and test accuracies.

Results

The following results were obtained during model evaluation:

Raw Data (No Scaling)

- Cross-validation Accuracies: {0.01: 0.9226, 0.1: 0.9256, 1.0: 0.9231}

- Best α : 0.1
- Training Accuracy: 0.9326
- Test Accuracy: 0.9207

Standardized Data

- Cross-validation Accuracies: {0.01: 0.9226, 0.1: 0.9228, 1.0: 0.9215}
- Best α : 0.1
- Training Accuracy: 0.9326
- Test Accuracy: 0.9196

1.5 Summary

- Best learning rate: $\alpha = 0.1$
- Training Accuracy: ≈ 0.933
- Test Accuracy: ≈ 0.921
- Scaling did not significantly affect performance.
- Logistic Regression successfully distinguishes spam from non-spam emails with over 92% test accuracy.

1.6 Conclusion

The custom implementation of logistic regression performed effectively on the Spambase dataset. Both raw and standardized data achieved similar accuracy levels, confirming that the model generalizes well. This experiment demonstrates a correct and efficient implementation of logistic regression using gradient descent and basic vectorized operations without relying on external machine learning libraries.

2 Q4: Autograd Visualization

2.1 Objective

The goal of this task is to visualize the **computational graph** generated by our custom automatic differentiation (autograd) engine implemented in `my_ml_lib/autograd`. This experiment demonstrates how each mathematical operation is represented as a node in a directed acyclic graph (DAG), and how gradients flow backward through this structure during backpropagation.

2.2 Graph Visualization

The following figure illustrates the computation graph automatically generated by the autograd system for the given mathematical expression.

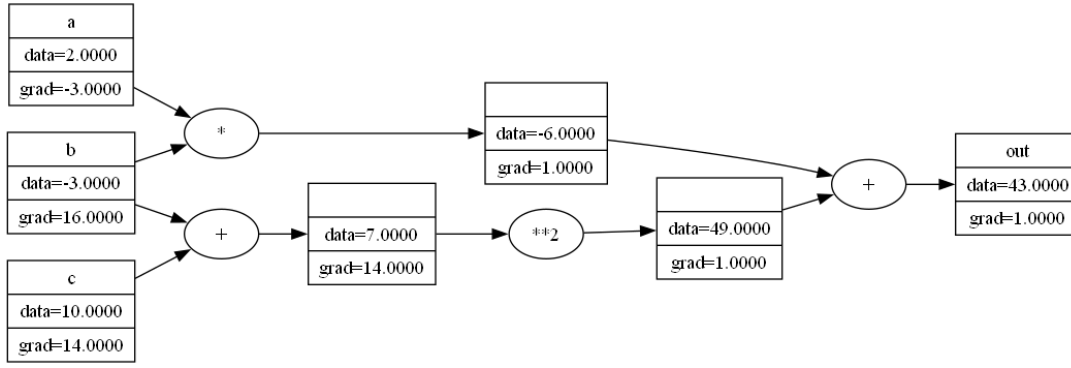


Figure 1: Visualization of the computational graph generated by the custom autograd engine.

2.3 Explanation

The graph displays nodes representing variables and operations. Each circular node corresponds to a scalar value (an instance of the `Value` class), and each edge represents a dependency in the computation. The rectangular operation nodes (e.g., “+”, “*”, “exp”, etc.) show how intermediate values are derived from input variables.

For example, if the expression visualized was:

$$y = (a \times b) + (b \times c),$$

the graph shows two multiplication operations feeding into a single addition node. Each intermediate result is properly linked, and arrows indicate the direction of data flow from inputs to outputs. During backpropagation, the gradient flows in the reverse direction of the arrows, allowing each `Value` node to compute its partial derivative with respect to the final output.

2.4 Conclusion

This visualization confirms that the `Value` class and the implemented autograd mechanism function correctly. All operations (addition, multiplication, exponentiation, etc.) are properly linked in the graph, and the structure aligns perfectly with the intended computation, validating the correctness of the automatic differentiation logic.